

15. Exercício 1 – Referências

Execute:

```
a = [10, 20, 30]
b = a

b.append(40)

print("a:", a)
print("b:", b)
print("id(a):", id(a))
print("id(b):", id(b))
"Qual será a saída?"
```

```
a: [10, 20, 30, 40]
b: [10, 20, 30, 40]
id(a): 135839619420096
id(b): 135839619420096
'Qual será a saída?'
```

2. Sim, a e b representam o mesmo objeto.
3. b = a cria outra referência para a mesma lista. Quando b.append(40) é executado a lista de "A" também é alterada.
4. Com b = a.copy(), seria criada uma nova lista, mas alterar b não muda a.

16. Exercício 2 – Construindo uma cadeia

Crie:

```
n1 = Node("A")
n2 = Node("B")
n3 = Node("C")
```

Conecte:

A → B → C → None

Depois percorra a estrutura e produza:

```
A
B
C
```

```
class Node:
    def __init__(self, valor):
        self.valor = valor
        self.proximo = None
```

```
n1 = Node("A")
n2 = Node("B")
n3 = Node("C")
```

```
n1.proximo = n2
n2.proximo = n3
n3.proximo = None
```

atual = n1

```
while atual is not None:
    print(atual.valor)
    atual = atual.proximo
```

```
A
B
C
```

17. Exercício 3 – Depuração

Análise:

```
n1 = Node(10)
n2 = Node(20)
n3 = Node(30)
```

```
n1.proximo = n2
n2.proximo = n3
```

atual = n1

```
while atual is not None:
    print(atual.valor)
```

Perguntas:

Qual é o problema?

Por que o programa não termina?

Qual linha deve ser acrescentada?

Qual será a saída depois da correção?

17. Exercício 3 — Depuração

1 - Problema: falta atualizar a variável atual.

2 - O programa não termina porque atual continua apontando para n1 e o while fica infinito.

3 - Deve ser acrescentada:

atual = atual.proximo

Saída:

10 20 30

18 - 22. Problema real — Sistema de atendimento de uma clínica

Uma clínica recebe pacientes durante o dia. Cada paciente possui:

nome;

idade;

prioridade.

Exemplo:

Ana — 32 anos — Normal

Bruno — 70 anos — Prioridade

Carlos — 45 anos — Normal

O sistema deve permitir:

adicionar paciente;

listar pacientes;

atender o primeiro paciente;

verificar se a fila está vazia;

informar a quantidade de pacientes.

```
class Paciente:
    def __init__(self, nome, idade, prioridade):
        self.nome = nome
        self.idade = idade
        self.prioridade = prioridade

class Node:
    def __init__(self, paciente):
        self.paciente = paciente
        self.proximo = None

class FilaAtendimento:
    def __init__(self):
        self.inicio = None
        self.fim = None
        self._tamanho = 0

    def adicionar(self, paciente):
        novo = Node(paciente)

        if self.esta_vazia():
            self.inicio = novo
            self.fim = novo
        else:
            self.fim.proximo = novo
            self.fim = novo

        self._tamanho += 1

    def atender(self):
        if self.esta_vazia():
            return None
```

```

    paciente = self.inicio.paciente
    self.inicio = self.inicio.proximo

    if self.inicio is None:
        self.fim = None

    self._tamanho -= 1
    return paciente

def listar(self):
    atual = self.inicio

    while atual is not None:
        p = atual.paciente
        print(f"{p.nome} - {p.idade} anos - {p.prioridade}")
        atual = atual.proximo

def esta_vazia(self):
    return self.inicio is None

def tamanho(self):
    return self._tamanho

fila = FilaAtendimento()

fila.adicionar(Paciente("Ana", 32, "Normal"))
fila.adicionar(Paciente("Bruno", 70, "Prioridade"))
fila.adicionar(Paciente("Carlos", 45, "Normal"))

fila.listar()

paciente = fila.atender()
print("Atendendo:", paciente.nome)

```

```

Ana - 32 anos - Normal
Bruno - 70 anos - Prioridade
Carlos - 45 anos - Normal
Atendendo: Ana

```

23 - Desafio — Atendimento prioritário

Solução: pacientes com prioridade serão colocados antes dos pacientes normais. Se já houver pacientes prioritários, o novo prioritário entra depois deles.

Estrutura:

Carlos (Prioridade) → Ana (Normal) → Bruno (Normal) → None

Local de inserção: o paciente prioritário deve ser inserido no início ou depois do último paciente prioritário. Implementação:

```
def adicionar(self, paciente): novo = Node(paciente)
```

```

if self.esta_vazia():
    self.inicio = novo
    self.fim = novo

elif paciente.prioridade == "Prioridade":
    if self.inicio.paciente.prioridade != "Prioridade":
        novo.proximo = self.inicio
        self.inicio = novo
    else:
        atual = self.inicio

        while (atual.proximo is not None and
               atual.proximo.paciente.prioridade == "Prioridade"):
            atual = atual.proximo

        novo.proximo = atual.proximo
        atual.proximo = novo

        if novo.proximo is None:
            self.fim = novo
else:
    self.fim.proximo = novo
    self.fim = novo

self._tamanho += 1

```

